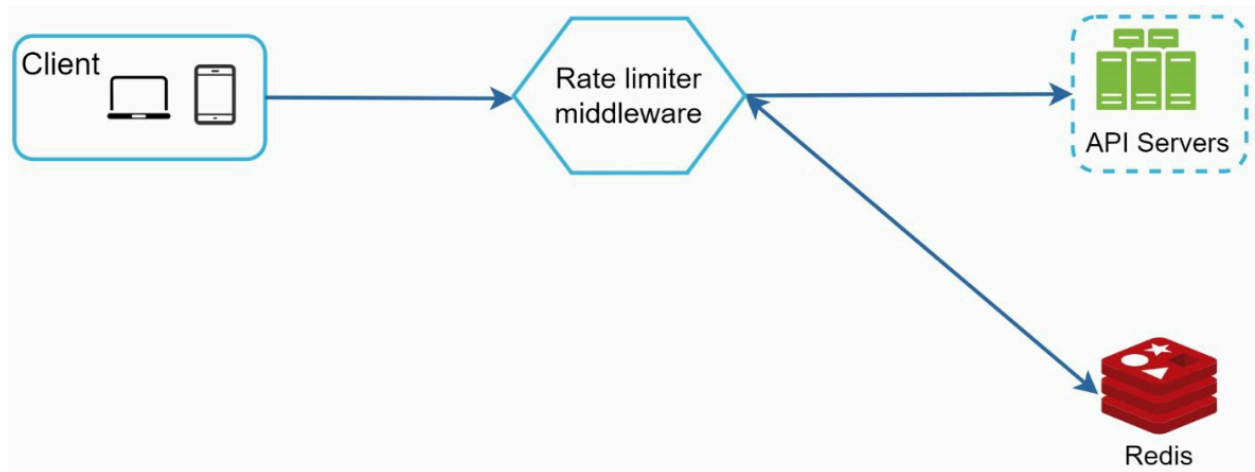


Design a Rate Limiter - Step 3: Design Deep Dive



The high-level design (above figure) does not answer some important questions:

- How are rate limiting rules created and stored?
- How are rate-limited requests handled?

1. Rate Limiting Rules

Rate limiting rules define **who can do what and how often**.

Example 1

None

domain: messaging

descriptors:

- key: message_type

Value: marketing

rate_limit:

unit: day

requests_per_unit: 5

Meaning:

- Maximum **5 marketing messages per day**

Example 2

None

```
domain: auth
```

```
descriptors:
```

```
- key: auth_type
```

```
  Value: login
```

```
  rate_limit:
```

```
    unit: minute
```

```
    requests_per_unit: 5
```

Meaning:

- Maximum **5 login attempts per minute**

Storage of Rules

- Rules are written in **configuration files**
- Stored on **disk**

2. Exceeding the Rate Limit

When a request exceeds the defined limit:

None

HTTP 429 → Too Many Requests

Handling Strategies

- Reject request immediately
- Optionally **enqueue requests** for later processing

Example:

- Orders during system overload can be **queued and processed later**

3. Rate Limiter Headers

Clients need visibility into their rate limits.
This is achieved via HTTP response headers.

Important Headers

None

`X-RateLimit-Remaining` → Remaining requests in current window

`X-RateLimit-Limit` → Max allowed requests per window

`X-RateLimit-Retry-After` → Time (in seconds) to wait before retry

When Limit is Exceeded

- Response:

None

HTTP 429 → Too Many Requests

-

Header included:

None

X-RateLimit-Retry-After

4. Key Takeaway

Deep dive focuses on:

- Defining flexible **rate limiting rules**
- Handling **exceeded requests**
- Providing **feedback to clients via headers**